

Explication du projet SIM_NN

Contrôle par réseau de neurones des phases d'un Stacked Intelligent Metasurface (SIM) — document de synthèse basé sur le fichier **SIM_NN_Phase_bis.py** et les fichiers associés du dossier **SIM_NN**.

1. Contexte general

Le SIM (Stacked Intelligent Metasurface) est un empilement de L metasurfaces reconfigurables placées entre une station de base (BS) et K utilisateurs (UE), fonctionnant en ondes millimétriques (28 GHz). Chaque metasurface est composée de N meta-atomes dont le déphasage peut être contrôlé numériquement (quantifié sur N_{bitQ} bits). Le but du script est d'entraîner un réseau de neurones qui, à partir de la matrice de canal H , prédit les déphasages à appliquer sur chaque meta-atome de chaque couche afin de maximiser l'efficacité spectrale (SE) du pire utilisateur (critère max-min fairness).

2. Structure de SIM_NN_Phase_bis.py

2.1 Fonctions clés

- **NormP_Q(sig)** : quantifie un signal de phase sur N_{bitQ} bits (2 bits ici, soit 4 niveaux discrets) via `tf.quantization.fake_quant_with_min_max_args`.
- **CustomLoss(y_true, y_pred)** : fonction de perte physique, cœur du modèle. Elle reconstruit la matrice de canal complexe à partir de `y_true` (partie réelle/imaginaire concaténées), quantifie les phases prédites, calcule le déphasage complexe $\exp(j\theta)$, propage le signal à travers les L couches du SIM (matrice G = produit cumulé des matrices diagonales de phase et des matrices de transmission W_l), puis calcule le débit ($\log_2(1+\text{SINR})$) de chaque utilisateur. La perte retournée est l'opposé du débit minimum moyen sur le batch (max-min fairness).

2.2 Paramètres physiques

Définis en dur en début de script : $K=4$ utilisateurs, $M=K$ antennes BS, $L=10$ metasurfaces, $N = N_x \times N_y = 7 \times 7 = 49$ meta-atomes par couche, $N_{\text{bitQ}}=2$ bits de quantification de phase, $f=28$ GHz, budget de puissance, bruit thermique (σ_{2k}), et modèle de path-loss (exposant $\bar{n}=3.5$).

2.3 Géométrie et matrices de transmission

À partir des paramètres physiques, le script calcule les distances entre meta-atomes (D_{nn}), entre couches successives (D_l), les angles d'incidence, puis construit les matrices de transmission inter-couches **W_l** et la matrice BS $\rightarrow$ première couche **W_1** (fonctions définies dans `functions.py`). Ces matrices modélisent physiquement la propagation de l'onde à travers l'empilement de metasurfaces.

2.4 Génération des données

Une (ou plusieurs) réalisation(s) de canal H sont générées aléatoirement (Rayleigh, corrélées spatialement via R_{nn}). Le canal est mis sous forme réelle/imaginaire concaténée pour servir d'entrée ET de "cible" (`y_true`) au modèle Keras : c'est une astuce pour faire transiter la matrice de canal jusqu'à la fonction de perte personnalisée, qui la reconstruit par reshape. L'apprentissage est donc **non supervisé** (pas de vraies étiquettes, la perte est un objectif physique directement calculé).

2.5 Architecture du réseau

Un CNN 1D : Conv1D(16 filtres, kernel=3, ReLU) → MaxPooling1D → Flatten → Dense(L*N, activation = fonction personnalisée mappant vers $[0, 2\pi]$). La sortie représente les L*N déphasages à appliquer sur l'ensemble des meta-atomes du SIM.

2.6 Entraînement, sauvegarde et évaluation

Le modèle est compilé avec l'optimiseur Adam et la CustomLoss, puis entraîné sur 1000 epochs (batch_size = btch). Les poids de chaque epoch sont stockés (callback), le meilleur epoch (perte minimale) est sélectionné, et les poids + le canal H sont sauvegardés dans un fichier .mat. Le modèle est ensuite réévalué : les phases prédites sont quantifiées, le débit par utilisateur est recalculé, et une courbe de fonction de répartition (CDF) de l'efficacité spectrale est tracée.

Point important : dans ce fichier, Nch = 1 et btch = 1, donc une seule réalisation de canal sert à la fois pour l'entraînement, la validation (H2 = H1) et le test (H3 = H1). Il ne s'agit donc pas d'une vraie généralisation statistique mais d'un test de convergence sur un seul point de canal.

3. Comment tester le code

- Verifier les dependances : tensorflow / keras, numpy, scipy, matplotlib.
- Se placer dans le dossier SIM_NN/ (le script fait `from functions import *`, donc `functions.py` doit etre accessible).
- Lancer : `python SIM_NN_Phase_bis.py`
- Pour un premier test rapide, reduire temporairement epochs (ex. 20-50 au lieu de 1000) afin de verifier l'absence d'erreurs avant un run complet, qui peut etre lent (boucles Python non vectorisees dans CustomLoss).
- Sorties attendues : résumé du modele dans la console, courbe de perte (train/val), fichier .mat contenant canal + poids, et courbe CDF de l'efficacite spectrale.
- Pour un test plus rigoureux : augmenter Nch (plusieurs realisations de canal) et generer H2/H3 independamment de H1 (actuellement ces lignes sont commentees dans le script).

4. Vue d'ensemble des fichiers du dossier SIM_NN

Fichier	Role
functions.py	Bibliotheque partagee : geometrie du SIM (distances, matrices de transmission W/W_1), path-loss
SIM_NN_Phase.py	Version proche de la variante "bis", controle uniquement les phases ($L=10$), option SGD commentee
SIM_NN_Phase_bis.py	Fichier de reference de ce document : variante "bis" (iteration corrigee/nettoyee), utilise un CNN
SIM_NN_Phase_Power.py	Etend le modele a deux sorties : task1_output (phases) + task2_output (puissances, via custom_
SIM_NN_Phase_StatCl.py	Variante avec $L=5$ (au lieu de 10), destinee a tourner avec plusieurs realisations de canal (Nch >
SIM_NN_Phase_Sys2.py	Variante pour une autre configuration systeme (autre parametrage d'antennes/utilisateurs).
ChannelEstim_NN.py / ChannelEstim_NN_Sys2.py	Reseaux medies a l'estimation de canal (probablement a partir de pilotes), distincts de l'optimisati
ANN_SIM_DualTask.py (+ .txt)	Modele "dual-task" plus general ; le .txt est probablement une sauvegarde/sortie texte associee.

En resume : `functions.py` est le module physique commun, les fichiers `SIM_NN_Phase*.py` sont des variantes experimentales du meme pipeline (phases seules, phases+puissance, autre systeme, autre nombre de couches), tandis que `ChannelEstim_NN*` et `ANN_SIM_DualTask` explorent des taches connexes (estimation de canal, apprentissage multi-tache).